Algorithms for Deciding the Safety of States in Fully Observable Non-deterministic Problems

Johannes Schmalz, Chaahat Jain

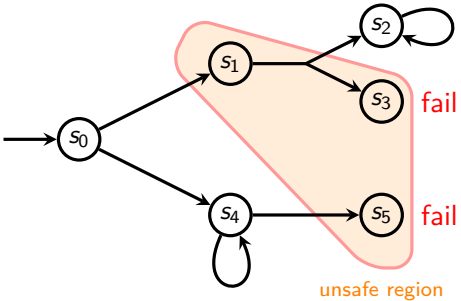
Foundations of AI
Universität des Saarlandes

ICAPS 2026

Setting

We consider Fully Observable Non-Deterministic (FOND) planning but with **fail states** instead of goal states — solutions are policies $\pi : S \to A$ that avoid fail states

A state is unsafe if reaching fail state is unavoidable.



Given a state, we want to decide if it is unsafe or not.

Some Context

Our problems are encoded in JANI and may have exponentially-many initial states. We want to decide if states are unsafe to find faults in learned policies.

Current Status

Jain et al. [2025] tested VI, iLAO*, LRTDP, and **TarjanSafe**.

☞ TarjanSafe worked very well.

New contributions

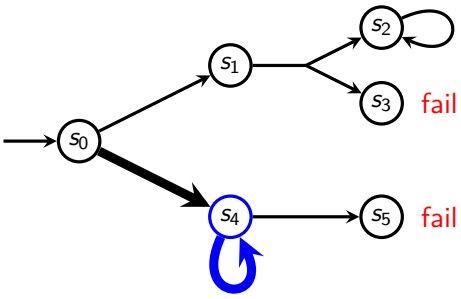
- TarjanSafe has an exponential worst-case behaviour w.r.t. graph
- we give a polynomial alternative

TarjanSafe [Jain et al., 2025]

DFS with SCC detection á la Tarjan's Algorithm

Marks SCC safe when popping SCC root and none if its elements are unsafe

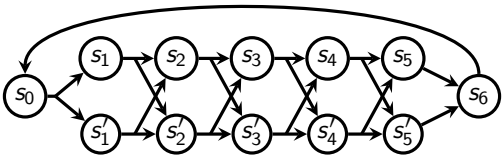
---



TarjanSafe [Jain et al., 2025]

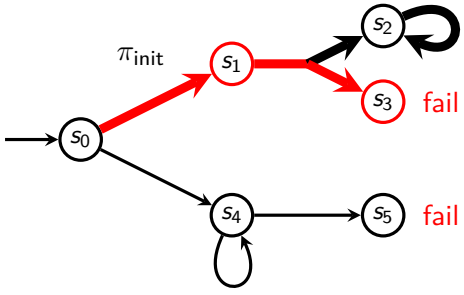
- best case $\Theta(|\pi_{\text{min-safe}}|)$
- worst case $\Omega(2^m)$

$m = |\mathrm{A}|$, $|\pi_{\text{min-safe}}|$ is a min. safe policy size *(if safe π exists)*



Naïve and Improved Policy Iteration (nPI and iPI)

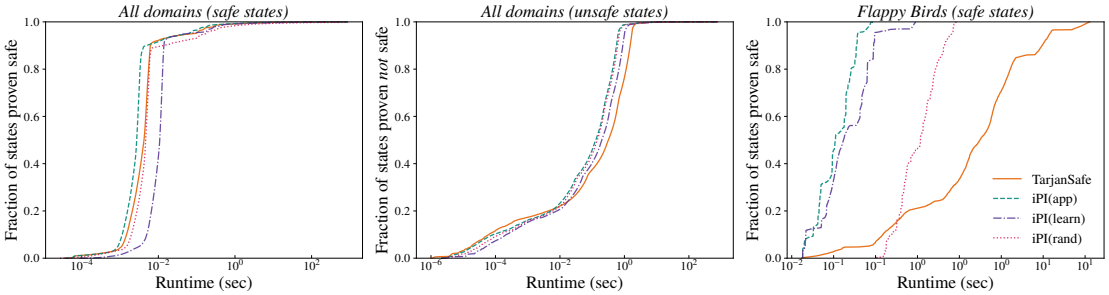
Start with π_{init}, check it with DFS, propagate issues and update π to fix them, repeat.



- best case $\Theta(|\pi_{\text{min-safe}}|)$
- worst case $O(bmn)$

$m = |\mathrm{A}|$, $n = |\mathrm{S}|$,
$b =$ non-det. branching,
$|\pi_{\text{min-safe}}|$ is a min. safe policy size

- **Domains**: blocksworld, transport, control benchmarks (e.g., inverted pendulum), *Flappy Bird*
- **Algorithms**: TarjanSafe and iPI with different π_{init}



- **Over "All domains"**: similar performance (these are TarjanSafe-friendly domains)
- **Over "Flappy Bird"**: exponential gap (for deciding safe states)
- **Different π_{init}**: difference is probably implementation

Summary

- **Setting:** decide safety of states in FOND
- TarjanSafe fast in practice but exponential worst-case runtime
- our algorithm (iPI) is fast in practice and polynomial worst-case!

In the paper we also look at **Unsafety Propagation**, connection between Goal-Reaching and State-Avoiding FOND, and a bit more

---

For more: `schmalz.cc/fond-fault-analysis`

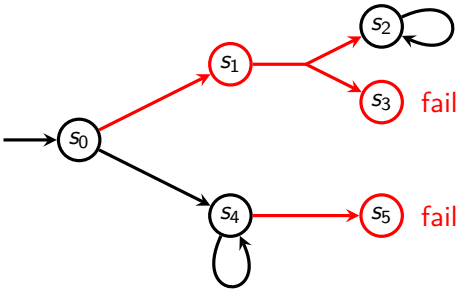
Cascioli, L.; Jain, C.; Steinmetz, M.; Davis, J.; and Hoffmann, J. 2025. Safety Debugging of Tree-Ensemble Action Policies in AI Planning: From Fault Detection to Fault Fixing. In *Proc. of Workshop on Reliability In Planning and Learning (RIPL)*.

Diekert, V.; and Kufleitner, M. 2022. Reachability Games and Parity Games. In *Int. Colloquium on Theoretical Aspects of Computing (ICTAC)*.

Jain, C.; Sherbakov, D.; Vinzent, M.; Steinmetz, M.; Davis, J.; and Hoffmann, J. 2025. Policy Safety Testing in Non-Deterministic Planning: Fuzzing, Test Oracles, Fault Analysis. In *Proc. of European Conf. on Artificial Intelligence (ECAI)*.

Propagate Unsafe Region based on [Diekert and Kufleitner, 2022]

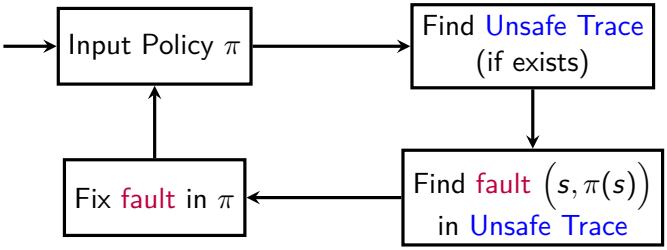
Mark fail states as unsafe, then iteratively mark (s, a) as unsafe if some successor is unsafe, and mark s unsafe if all its actions unsafe



- best and worst case $\Theta(|\mathcal{Y}|)$

$\mathcal{Y}$ are state-action pairs with effect in unsafe region

Context: Policy Repair Loop [Cascioli et al., 2025]



$\left(s, \pi(s)\right)$ is a fault if s is safe but $\pi(s)$ leads to unsafe

Context: Problem Encoding

We use JANI and we may have exponentially-many initial states.